



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУК4 Программное обеспечение ЭВМ, информационные технологии

ЛАБОРАТОРНАЯ РАБОТА №3

«Работа с матрицами и преобразованиями в OpenGL»

по дисциплине: *«Компьютерная графика»*

Выполнил: студент группы ИУК4-43Б

(Подпись)

Афанасьева Ю.Р.

(И.О. Фамилия)

Проверил:

(Подпись)

Тихонович Е.О.

(И.О. Фамилия)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:

- Оценка:

Цель: формирование практических навыков по работе с матричными преобразованиями для изменения сцены в целом и отдельных её объектов, закрепление знаний о способах проецирования, изучения методов работы со стеком матриц средствами OpenGL, создание ряда многообъектных сцен и их преобразование с использованием переноса, поворота и масштабирования.

Задачи:

сформировать понимание преобразований наблюдения, модели и проектирования, познакомиться с концепцией матриц преобразования, выяснить назначение единичной матрицы, научиться создавать приложения OpenGL с использованием матриц преобразования и ранее изученных объектов.

Вариант 3

Задание выполняется согласно варианту. По завершении готовится отчет.

- 1) [Листинг 1](#) преобразовать согласно варианту.
- 2) С помощью [Листинга 2](#) и [Листинга 3](#) изобразить фигуру согласно варианту с разным цветом внутренних и внешних граней (возможен разный цвет всех внешних граней). Поменять цвет фона.
- 3) Для [Листинга 4](#) реализовать изменения согласно варианту.

Этапы выполнения работы:

1. Был изучен принцип использования матриц преобразования OpenGL, функций переноса и поворота объектов, стека матриц, а также ортогографической и перспективной проекций.
2. На основе листинга 1 была создана анимированная модель атома. В соответствии с вариантом 3 круговые траектории электронов были заменены замкнутыми эллиптическими. Координаты электронов рассчитывались с помощью функций $\cos f()$ и $\sin f()$. Для периодического обновления сцены использовался таймер GLUT.
3. На основе листингов 2 и 3 была построена объёмная полая фигура. В листинге 2 использовалась ортогографическая проекция, а в листинге 3 перспективная. Внешние грани фигуры были окрашены в синий цвет, а внутренние в оранжевый. Цвет фона был изменён на светло-серый. Была сохранена возможность вращения фигуры клавишами со стрелками.
4. На основе листинга 4 была создана анимированная модель системы «Солнце — Земля — Луна». По заданию в систему была добавлена ещё одна планета без спутника. Новая планета движется в той же плоскости и по той же траектории, что Земля, и смещена относительно неё на 180 градусов.

Задание 1:

Изменить круговые орбиты на произвольные замкнутые.

Листинг 1

```
#include "glut.h"
#include <math.h>

// Величина поворота
static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;

// Вызывается для рисования сцены
void RenderScene(void)
{
    // Угол поворота вокруг ядра
    static GLfloat fElect1 = 0.0f;
    GLfloat angle = fElect1 * 3.14159265f / 180.0f;

    // Очищаем окно текущим цветом очистки
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // Обновляем матрицу наблюдения модели
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();

    // Транслируем всю сцену в поле зрения
    // Это исходное преобразование наблюдения
    glTranslatef(0.0f, 0.0f, -100.0f);

    // Красное ядро
    glColor3ub(255, 0, 0);
    glutSolidSphere(10.0f, 15, 15);

    // Желтые электроны
    glColor3ub(255, 255, 0);

    // Орбита первого электрона
    // Записываем преобразование наблюдения
    glPushMatrix();

    // Движение по замкнутой эллиптической орбите
    glTranslatef(90.0f * cosf(angle), 0.0f, 50.0f * sinf(angle));

    // Рисуем электрон
    glutSolidSphere(6.0f, 15, 15);

    // Восстанавливаем преобразование наблюдения
    glPopMatrix();

    // Орбита второго электрона
```

```

    glPushMatrix();
    glRotatef(45.0f, 0.0f, 0.0f, 1.0f);
    glTranslatef(70.0f * cosf(angle), 0.0f, 40.0f * sinf(angle));
    glutSolidSphere(6.0f, 15, 15);
    glPopMatrix();

    // Орбита третьего электрона
    glPushMatrix();
    glRotatef(360.0f - 45.0f, 0.0f, 0.0f, 1.0f);
    glTranslatef(60.0f * cosf(angle), 0.0f, 35.0f * sinf(angle));
    glutSolidSphere(6.0f, 15, 15);
    glPopMatrix();

    // Увеличиваем угол поворота
    fElect1 += 10.0f;
    if (fElect1 > 360.0f)
        fElect1 = 0.0f;

    // Показываем построенное изображение
    glutSwapBuffers();
}

// Функция выполняет необходимую инициализацию
// в контексте визуализации.
void SetupRC()
{
    glEnable(GL_DEPTH_TEST); // Удаление скрытых поверхностей
    glFrontFace(GL_CCW);    // Полигоны с обходом против часовой стрелки
                             // направлены наружу
    glEnable(GL_CULL_FACE); // Внутри пирамиды расчеты не производятся

    // Черный фон
    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;

    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;

    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;

    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    if (key > 356.0f)
        xRot = 0.0f;
}

```

```

    if (key < -1.0f)
        xRot = 355.0f;

    if (key > 356.0f)
        yRot = 0.0f;

    if (key < -1.0f)
        yRot = 355.0f;

    // Перерисовывает сцену с новыми координатами
    glutPostRedisplay();
}

void TimerFunc(int value)
{
    glutPostRedisplay();
    glutTimerFunc(100, TimerFunc, 1);
}

void ChangeSize(int w, int h)
{
    GLfloat nRange = 100.0f;

    // Предотвращение деления на ноль
    if (h == 0)
        h = 1;

    // Устанавливает поле просмотра по размерам окна
    glViewport(0, 0, w, h);

    // Обновляет стек матрицы проектирования
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();

    // Устанавливает объем отсечения с помощью отсекающих плоскостей
    // (левая, правая, нижняя, верхняя, ближняя, дальняя)
    if (w <= h)
        glOrtho(-nRange, nRange, nRange * h / w, -nRange * h / w,
                -nRange * 2.0f, nRange * 2.0f);
    else
        glOrtho(-nRange * w / h, nRange * w / h, nRange, -nRange,
                -nRange * 2.0f, nRange * 2.0f);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

int main(int argc, char *argv[])
{
    glutInit(&argc, argv);

```

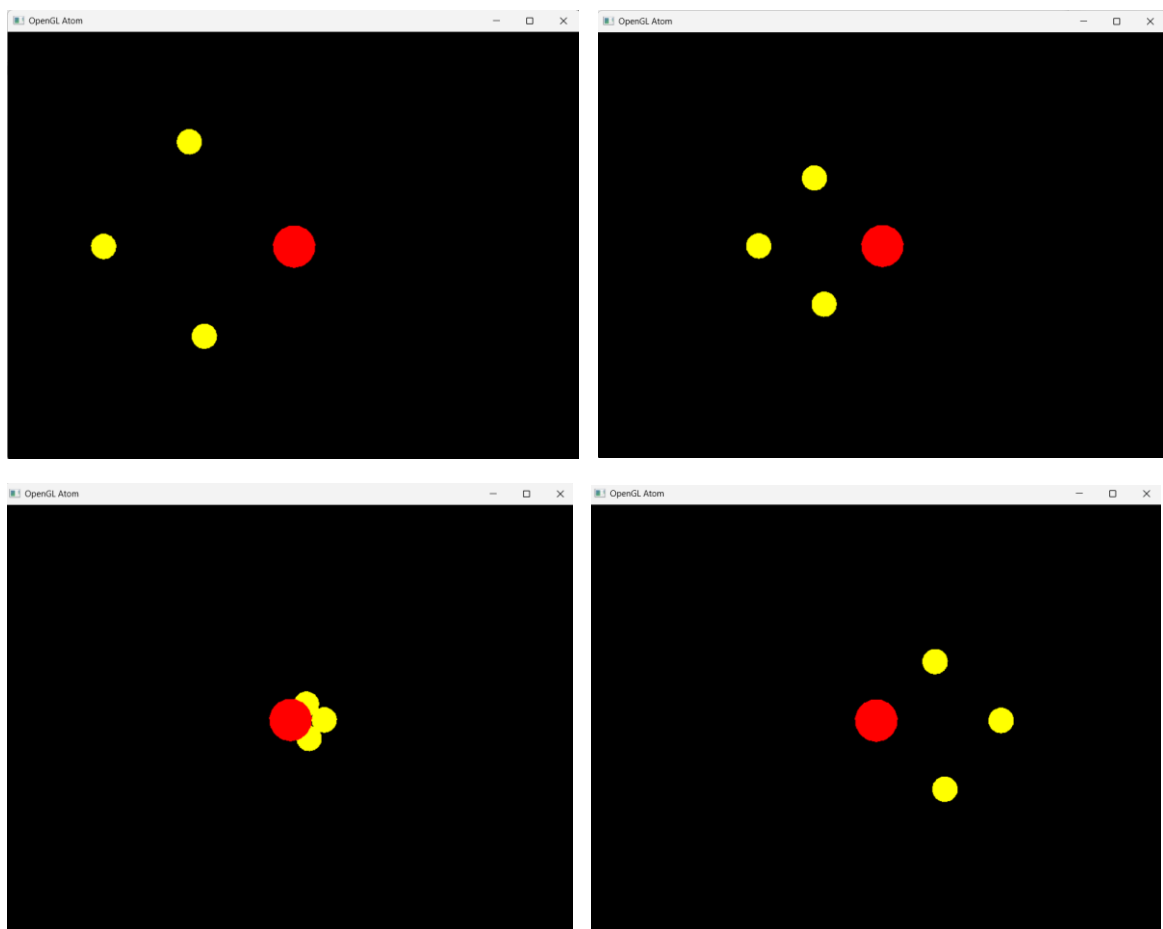
```

glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
glutInitWindowSize(800, 600);
glutCreateWindow("OpenGL Atom");
glutReshapeFunc(ChangeSize);
glutSpecialFunc(SpecialKeys);
glutDisplayFunc(RenderScene);
glutTimerFunc(500, TimerFunc, 1);
SetupRC();
glutMainLoop();

return 0;
}

```

Результаты работы программы 1



Задание 2: изобразить фигуру согласно варианту с разным цветом внутренних и внешних граней (возможен разный цвет всех внешних граней). Поменять цвет фона.



Листинг 2

```
#include "glut.h"

static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;

struct Point3
{
    GLfloat x;
    GLfloat y;
    GLfloat z;
};

void DrawQuad(const Point3 &a, const Point3 &b, const Point3 &c, const Point3 &d)
{
    GLfloat ux = b.x - a.x;
    GLfloat uy = b.y - a.y;
    GLfloat uz = b.z - a.z;
    GLfloat vx = c.x - a.x;
    GLfloat vy = c.y - a.y;
    GLfloat vz = c.z - a.z;

    glNormal3f(uy * vz - uz * vy,
               uz * vx - ux * vz,
               ux * vy - uy * vx);

    glVertex3f(a.x, a.y, a.z);
    glVertex3f(b.x, b.y, b.z);
    glVertex3f(c.x, c.y, c.z);
    glVertex3f(d.x, d.y, d.z);
}

void DrawFigure()
{
    const GLfloat frontZ = 45.0f;
    const GLfloat backZ = -45.0f;
```

```

const Point3 outerFront[6] = {
    {-110.0f, 0.0f, frontZ},
    {-80.0f, -40.0f, frontZ},
    {80.0f, -40.0f, frontZ},
    {110.0f, 0.0f, frontZ},
    {80.0f, 40.0f, frontZ},
    {-80.0f, 40.0f, frontZ}};

const Point3 outerBack[6] = {
    {-110.0f, 0.0f, backZ},
    {-80.0f, -40.0f, backZ},
    {80.0f, -40.0f, backZ},
    {110.0f, 0.0f, backZ},
    {80.0f, 40.0f, backZ},
    {-80.0f, 40.0f, backZ}};

const Point3 innerFront[6] = {
    {-82.0f, 0.0f, frontZ},
    {-60.0f, -22.0f, frontZ},
    {60.0f, -22.0f, frontZ},
    {82.0f, 0.0f, frontZ},
    {60.0f, 22.0f, frontZ},
    {-60.0f, 22.0f, frontZ}};

const Point3 innerBack[6] = {
    {-82.0f, 0.0f, backZ},
    {-60.0f, -22.0f, backZ},
    {60.0f, -22.0f, backZ},
    {82.0f, 0.0f, backZ},
    {60.0f, 22.0f, backZ},
    {-60.0f, 22.0f, backZ}};

glBegin(GL_QUADS);
// Внешние грани
glColor3f(0.15f, 0.45f, 0.90f);
for (int i = 0; i < 6; ++i)
{
    int next = (i + 1) % 6;
    // Передняя и задняя части рамки
    DrawQuad(outerFront[i], outerFront[next], innerFront[next],
innerFront[i]);
    DrawQuad(outerBack[next], outerBack[i], innerBack[i], innerBack[next]);
    // Наружная боковая поверхность
    DrawQuad(outerFront[next], outerFront[i], outerBack[i], outerBack[next]);
}

// Внутренние грани
glColor3f(1.0f, 0.45f, 0.10f);
for (int i = 0; i < 6; ++i)
{
    int next = (i + 1) % 6;

```



```

        DrawQuad(innerFront[i], innerFront[next], innerBack[next], innerBack[i]);
    }

    glEnd();
}

void ChangeSize(GLsizei w, GLsizei h)
{
    GLfloat nRange = 150.0f;

    if (h == 0)
        h = 1;
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    if (w <= h)
        glOrtho(-nRange, nRange, -nRange * h / w, nRange * h / w,
                -nRange * 2.0f, nRange * 2.0f);
    else
        glOrtho(-nRange * w / h, nRange * w / h, -nRange, nRange,
                -nRange * 2.0f, nRange * 2.0f);

    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void SetupRC()
{
    glEnable(GL_DEPTH_TEST);

    // Новый цвет фона
    glClearColor(0.82f, 0.86f, 0.90f, 1.0f);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    xRot = static_cast<GLfloat>(static_cast<int>(xRot) % 360);
    yRot = static_cast<GLfloat>(static_cast<int>(yRot) % 360);
    glutPostRedisplay();
}

void RenderScene()

```

```

{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glPushMatrix();
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);
    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    DrawFigure();
    glPopMatrix();

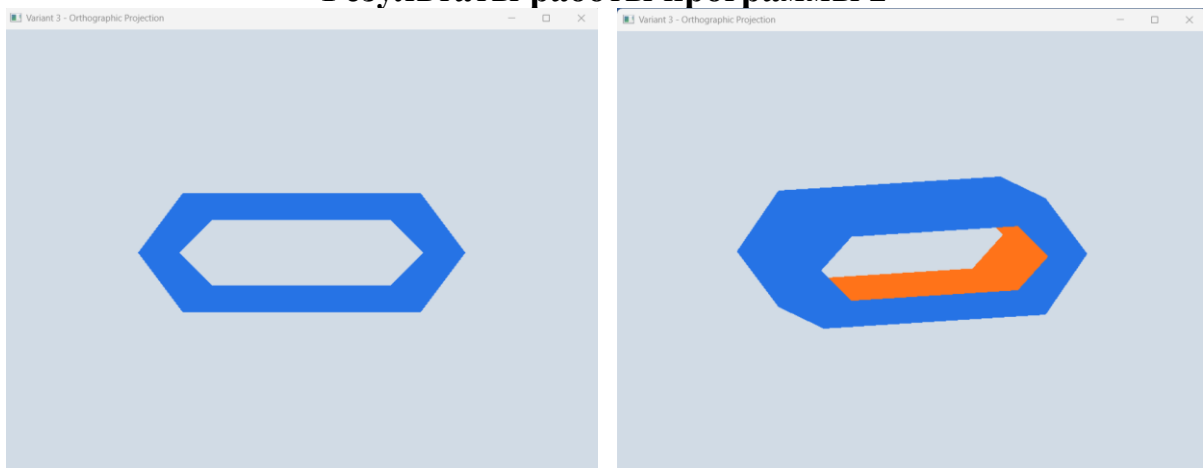
    glutSwapBuffers();
}

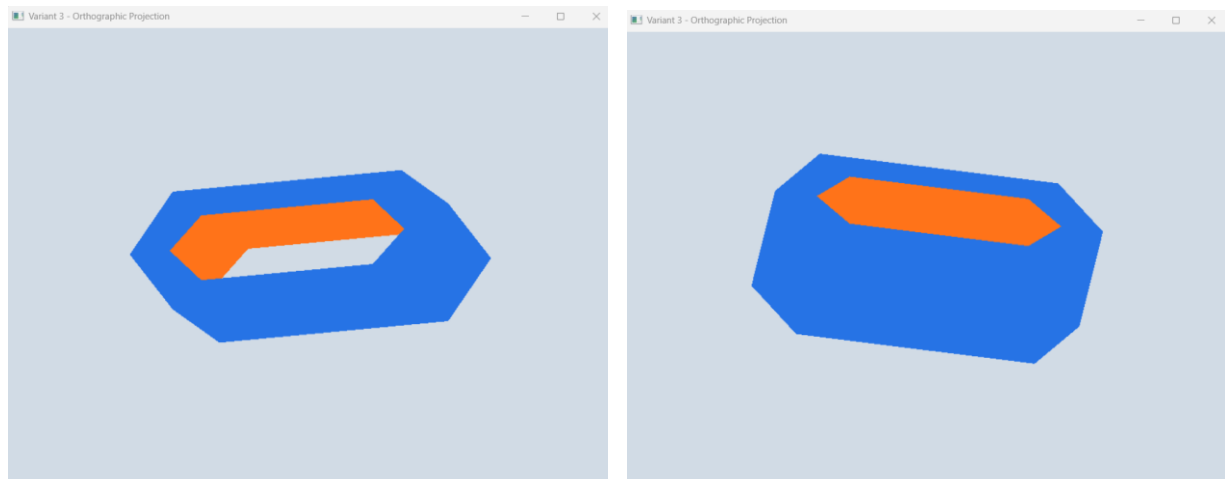
int main(int argc, char *argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Variant 3 - Orthographic Projection");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();

    return 0;
}

```

Результаты работы программы 2





Листинг 3

```
#include "glut.h"
#include <GL/glu.h>

static GLfloat xRot = 0.0f;
static GLfloat yRot = 0.0f;

struct Point3
{
    GLfloat x;
    GLfloat y;
    GLfloat z;
};

void DrawQuad(const Point3 &a, const Point3 &b, const Point3 &c, const Point3 &d)
{
    GLfloat ux = b.x - a.x;
    GLfloat uy = b.y - a.y;
    GLfloat uz = b.z - a.z;
    GLfloat vx = c.x - a.x;
    GLfloat vy = c.y - a.y;
    GLfloat vz = c.z - a.z;

    glNormal3f(uy * vz - uz * vy,
               uz * vx - ux * vz,
               ux * vy - uy * vx);

    glVertex3f(a.x, a.y, a.z);
    glVertex3f(b.x, b.y, b.z);
    glVertex3f(c.x, c.y, c.z);
    glVertex3f(d.x, d.y, d.z);
}

void DrawFigure()
{
    const GLfloat frontZ = 45.0f;
    const GLfloat backZ = -45.0f;
```

```

const Point3 outerFront[6] = {
    {-110.0f, 0.0f, frontZ},
    {-80.0f, -40.0f, frontZ},
    {80.0f, -40.0f, frontZ},
    {110.0f, 0.0f, frontZ},
    {80.0f, 40.0f, frontZ},
    {-80.0f, 40.0f, frontZ}};

const Point3 outerBack[6] = {
    {-110.0f, 0.0f, backZ},
    {-80.0f, -40.0f, backZ},
    {80.0f, -40.0f, backZ},
    {110.0f, 0.0f, backZ},
    {80.0f, 40.0f, backZ},
    {-80.0f, 40.0f, backZ}};

const Point3 innerFront[6] = {
    {-82.0f, 0.0f, frontZ},
    {-60.0f, -22.0f, frontZ},
    {60.0f, -22.0f, frontZ},
    {82.0f, 0.0f, frontZ},
    {60.0f, 22.0f, frontZ},
    {-60.0f, 22.0f, frontZ}};

const Point3 innerBack[6] = {
    {-82.0f, 0.0f, backZ},
    {-60.0f, -22.0f, backZ},
    {60.0f, -22.0f, backZ},
    {82.0f, 0.0f, backZ},
    {60.0f, 22.0f, backZ},
    {-60.0f, 22.0f, backZ}};

glBegin(GL_QUADS);

// Внешние грани
glColor3f(0.15f, 0.45f, 0.90f);
for (int i = 0; i < 6; ++i)
{
    int next = (i + 1) % 6;
    DrawQuad(outerFront[i], outerFront[next], innerFront[next],
innerFront[i]);
    DrawQuad(outerBack[next], outerBack[i], innerBack[i], innerBack[next]);
    DrawQuad(outerFront[next], outerFront[i], outerBack[i], outerBack[next]);
}

// Внутренние грани
glColor3f(1.0f, 0.45f, 0.10f);
for (int i = 0; i < 6; ++i)
{
    int next = (i + 1) % 6;
    DrawQuad(innerFront[i], innerFront[next], innerBack[next], innerBack[i]);

```

```

    }

    glEnd();
}

void ChangeSize(GLsizei w, GLsizei h)
{
    if (h == 0)
        h = 1;

    GLfloat fAspect = static_cast<GLfloat>(w) / static_cast<GLfloat>(h);

    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(60.0f, fAspect, 1.0f, 500.0f);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void SetupRC()
{
    glEnable(GL_DEPTH_TEST);

    // Новый цвет фона
    glClearColor(0.82f, 0.86f, 0.90f, 1.0f);
}

void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_UP)
        xRot -= 5.0f;
    if (key == GLUT_KEY_DOWN)
        xRot += 5.0f;
    if (key == GLUT_KEY_LEFT)
        yRot -= 5.0f;
    if (key == GLUT_KEY_RIGHT)
        yRot += 5.0f;

    xRot = static_cast<GLfloat>(static_cast<int>(xRot) % 360);
    yRot = static_cast<GLfloat>(static_cast<int>(yRot) % 360);
    glutPostRedisplay();
}

void RenderScene()
{
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glPushMatrix();
    glTranslatef(0.0f, 0.0f, -260.0f);
    glRotatef(xRot, 1.0f, 0.0f, 0.0f);

```

```

    glRotatef(yRot, 0.0f, 1.0f, 0.0f);
    DrawFigure();
    glPopMatrix();

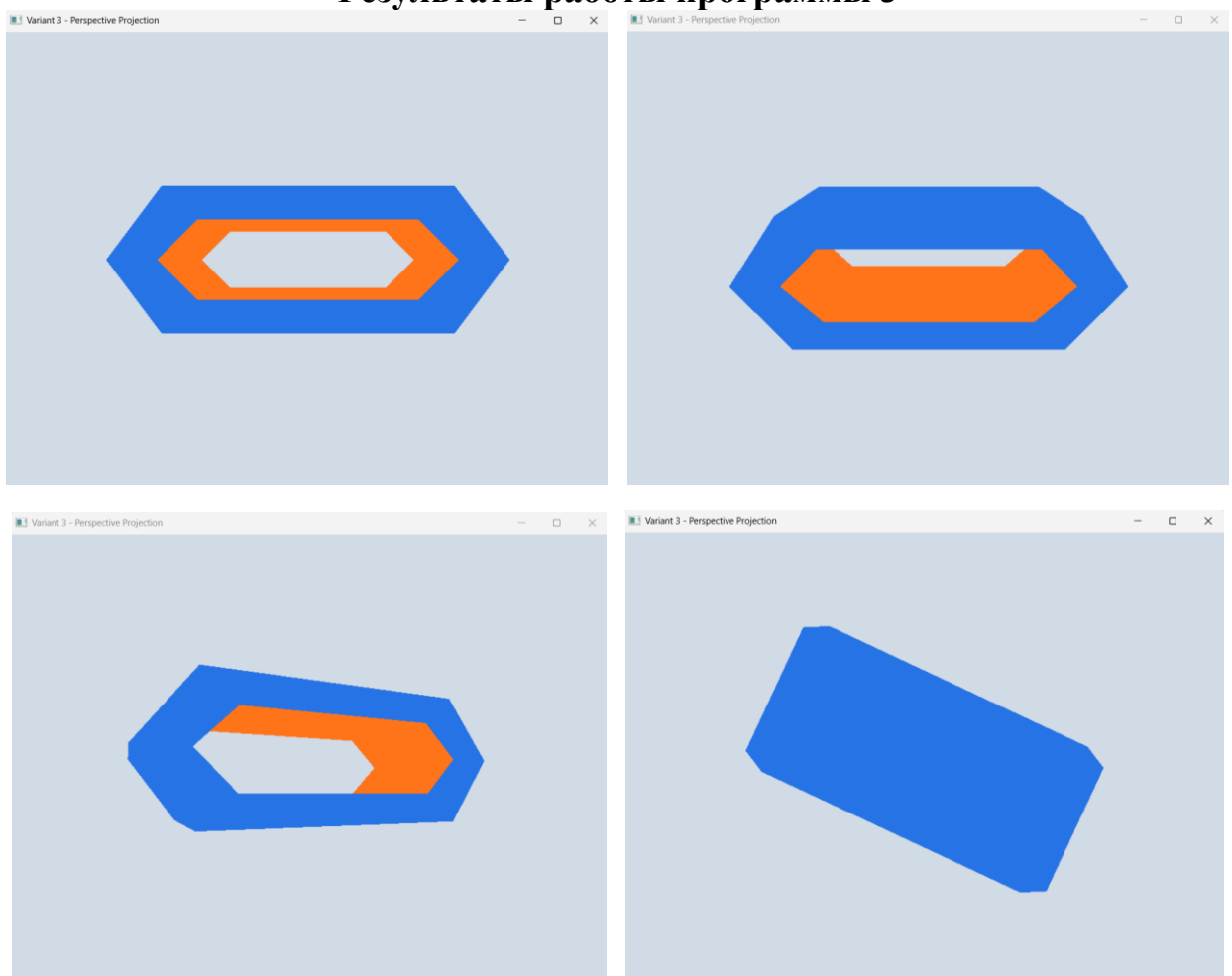
    glutSwapBuffers();
}

int main(int argc, char *argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Variant 3 - Perspective Projection");
    glutReshapeFunc(ChangeSize);
    glutSpecialFunc(SpecialKeys);
    glutDisplayFunc(RenderScene);
    SetupRC();
    glutMainLoop();

    return 0;
}

```

Результаты работы программы 3



Задание 3:

Добавить одну планету без спутника в плоскости и траектории Земля-Луна.

Листинг 4

```
#include "glut.h"
#include <GL/glu.h>

// Параметры освещения
GLfloat whiteLight[] = {0.2f, 0.2f, 0.2f, 1.0f};
GLfloat sourceLight[] = {0.8f, 0.8f, 0.8f, 1.0f};
GLfloat lightPos[] = {0.0f, 0.0f, 0.0f, 1.0f};

// Вызывается для рисования сцены
void RenderScene()
{
    // Углы поворота системы Земля-Луна
    static GLfloat fMoonRot = 0.0f;
    static GLfloat fEarthRot = 0.0f;

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    glMatrixMode(GL_MODELVIEW);
    glPushMatrix();

    // Перемещаем всю сцену в поле зрения
    glTranslatef(0.0f, 0.0f, -300.0f);

    // Солнце
    glDisable(GL_LIGHTING);
    glColor3ub(255, 255, 0);
    glutSolidSphere(15.0f, 30, 17);
    glEnable(GL_LIGHTING);

    // Источник света находится в центре Солнца
    glLightfv(GL_LIGHT0, GL_POSITION, lightPos);

    // Земля и Луна
    glPushMatrix();
    glRotatef(fEarthRot, 0.0f, 1.0f, 0.0f);
    glTranslatef(105.0f, 0.0f, 0.0f);

    // Земля
    glColor3ub(0, 0, 255);
    glutSolidSphere(15.0f, 30, 17);

    // Луна вращается относительно Земли
    glRotatef(fMoonRot, 0.0f, 1.0f, 0.0f);
    glTranslatef(30.0f, 0.0f, 0.0f);
    glColor3ub(200, 200, 200);
    glutSolidSphere(6.0f, 30, 17);
}
```

```

glPopMatrix();

// Дополнительная планета без спутника
// Она движется в плоскости и по траектории системы Земля-Луна
glPushMatrix();
glRotatef(fEarthRot + 180.0f, 0.0f, 1.0f, 0.0f);
glTranslatef(105.0f, 0.0f, 0.0f);
glColor3ub(0, 200, 100);
glutSolidSphere(12.0f, 30, 17);
glPopMatrix();

glPopMatrix();

// Изменяем углы поворота
fMoonRot += 15.0f;
if (fMoonRot > 360.0f)
    fMoonRot = 0.0f;

fEarthRot += 5.0f;
if (fEarthRot > 360.0f)
    fEarthRot = 0.0f;

glutSwapBuffers();
}

// Выполняет необходимую инициализацию визуализации
void SetupRC()
{
    glEnable(GL_DEPTH_TEST);
    glFrontFace(GL_CCW);
    glEnable(GL_CULL_FACE);

    glEnable(GL_LIGHTING);
    glLightModelfv(GL_LIGHT_MODEL_AMBIENT, whiteLight);
    glLightfv(GL_LIGHT0, GL_DIFFUSE, sourceLight);
    glLightfv(GL_LIGHT0, GL_POSITION, lightPos);
    glEnable(GL_LIGHT0);

    glEnable(GL_COLOR_MATERIAL);
    glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE);

    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
}

void TimerFunc(int value)
{
    glutPostRedisplay();
    glutTimerFunc(100, TimerFunc, 1);
}

void ChangeSize(int w, int h)

```



```

{
    if (h == 0)
        h = 1;

    GLfloat fAspect = static_cast<GLfloat>(w) / static_cast<GLfloat>(h);
    glViewport(0, 0, w, h);

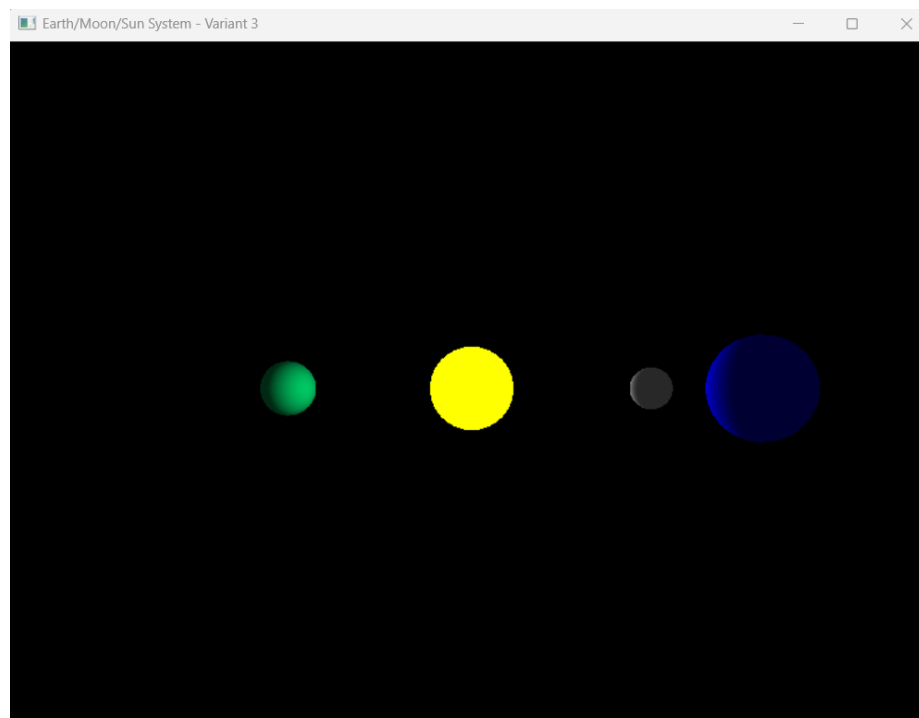
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0f, fAspect, 1.0f, 425.0f);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

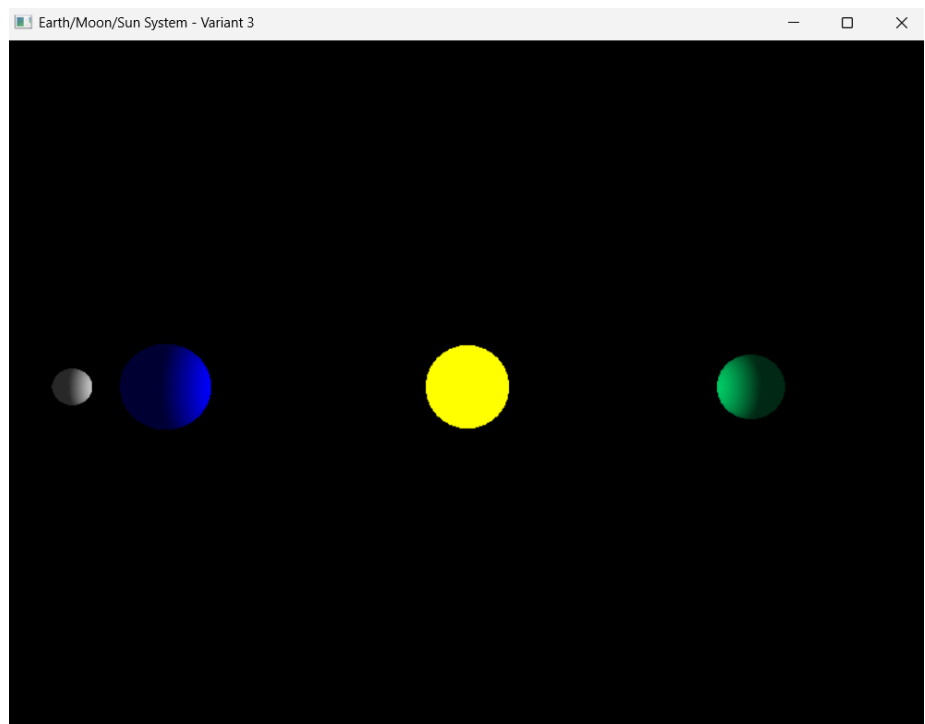
int main(int argc, char *argv[])
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(800, 600);
    glutCreateWindow("Earth/Moon/Sun System - Variant 3");
    glutReshapeFunc(ChangeSize);
    glutDisplayFunc(RenderScene);
    glutTimerFunc(250, TimerFunc, 1);
    SetupRC();
    glutMainLoop();

    return 0;
}

```

Результаты работы программы 4





Вывод: в ходе работы были получены практические навыки по работе с матричными преобразованиями для изменения сцены в целом и отдельных её объектов, закреплены знания о способах проецирования, изучены методы работы со стеком матриц средствами OpenGL, создан ряд многообъектных сцен и их преобразование с использованием переноса, поворота и масштабирования.